

Verba Dictionary Mobile Application — System Design Documentation

Client: LexiTech Solutions Ltd, Kigali City

Project: Cross-Platform Dictionary Mobile Application

API: Free Dictionary API — <https://api.dictionaryapi.dev/api/v2/entries/en/>

Tech Stack: React Native + Expo · TypeScript · Axios · React Navigation

Table of Contents

- 1. Data Flow Diagram (DFD) — Level 0 (Context)
- 2. Data Flow Diagram (DFD) — Level 1 (Detailed)
- 3. Application Architecture
- 4. Navigation Flow
- 5. API Endpoints
- 6. Required Pages / Screens
- 7. Component Hierarchy
- 8. Data Models (Entity Relationship)
- 9. State Management Architecture
- 10. Error Handling Flow

1. Data Flow Diagram — Level 0 (Context)

The Context DFD shows the **system boundary** of the Verba application and its interaction with external entities.

```
graph LR
    User((User))
    System[Verba Dictionary Mobile Application]
    API[Free Dictionary API Server]
    Storage[AsyncStorage Local Device]

    User -->|Search word Tap history Play audio Save word| System
    System -->|Definitions Phonetics Audio playback Error messages| User
    System -->|GET /api/v2/entries/en/{word}| API
    API -->|JSON response word data 404 error| System
    System -->|Save history Save bookmarks Save settings| Storage
    Storage -->|Load history Load bookmarks Load preferences| System

    style System fill:#4F46E5,color:#fff,stroke:#3730A3,stroke-width:2px
    style API fill:#0891B2,color:#fff,stroke:#0E7490,stroke-width:2px
    style Storage fill:#D97706,color:#fff,stroke:#B45309,stroke-width:2px
    style User fill:#7C3AED,color:#fff,stroke:#6D28D9,stroke-width:2px
```

Entities Explained

Entity	Type	Description
User	External	End-user on Android/iOS device
Verba Application	Process	The React Native mobile app (system boundary)
Free Dictionary API	External	REST API providing word data at <code>dictionaryapi.dev</code>
AsyncStorage	Data Store	On-device persistent storage for history, saved words, settings

2. Data Flow Diagram — Level 1 (Detailed)

This decomposes the system into its internal processes, showing how data flows between each Activity defined in the Integrated Situation.



Process Descriptions

Process	Activity	Description
1.0	Activity 1	Validates input (non-empty check), formats word (trim, lowercase)
2.0	Activity 1	Sends Axios GET to API, parses JSON via <code>apiParser.ts</code> , handles HTTP errors
3.0	Activity 2	Extracts word, phonetics, meanings, definitions, examples for display
4.0	Activity 3	Loads audio URL via <code>expo-av</code> , manages play/pause/stop states
5.0	Activity 4	Drawer menu renders history, allows re-search by tapping items
6.0	Activity 5	Detects 404, timeout, network errors; shows appropriate error screens
7.0	Extended	Saves words to collections (Favorites/Academic/Travel), tracks mastery

3. Application Architecture

The layered architecture showing the separation of concerns from UI to data.

```

graph TD
    subgraph Presentation["🖥️ PRESENTATION LAYER"]
        direction LR
        Screens["Screens\n(7 screens)"]
        Components["Reusable Components\n(10 components)"]
        Navigation["React Navigation\n(Stack + Drawer)"]
    end

    subgraph State["🧠 STATE MANAGEMENT LAYER"]
        direction LR
        ThemeCtx["ThemeContext\n(colors, font size)"]
        HistoryCtx["HistoryContext\n(search history)"]
        SavedCtx["SavedContext\n(bookmarks, streak)"]
    end

    subgraph Business["⚙️ BUSINESS LOGIC LAYER"]
        direction LR
        Hooks["Custom Hooks\nuseAudio · useDebounce"]
        Utils["Utilities\napiParser · formatters"]
        Models["TypeScript Models\nDictionaryEntry\nSavedWordItem\nSearchHistoryItem"]
    end

    subgraph Data["📁 DATA ACCESS LAYER"]
        direction LR
        AxiosClient["Axios HTTP Client\n(api.ts)"]
        DictService["dictionaryService.ts\nlookupWord()"]
        AsyncStore["AsyncStorage\n(persistence)"]
    end

    subgraph External["🌐 EXTERNAL"]
        DictAPI["Free Dictionary API\nhttps://api.dictionaryapi.dev"]
        AudioCDN["Audio CDN\n(MP3 pronunciations)"]
    end

    Screens --> Components
    Screens --> Navigation
    Screens --> State
    Components --> State
    State --> Business
    Business --> Data
    Data --> External

    style Presentation fill:#EEF2FF,stroke:#4F46E5,stroke-width:2px,color:#1E1B4B
    style State fill:#F0FDF4,stroke:#16A34A,stroke-width:2px,color:#14532D
    style Business fill:#FFF7ED,stroke:#EA580C,stroke-width:2px,color:#7C2D12
    style Data fill:#FEF2F2,stroke:#DC2626,stroke-width:2px,color:#7F1D1D
    style External fill:#F0F9FF,stroke:#0284C7,stroke-width:2px,color:#0C4A6E

```

Layer Responsibilities

Layer	Responsibility	Key Files
Presentation	UI rendering, user interaction, screen layout	*Screen.tsx , *Component.tsx , AppNavigator.tsx
State	Global app state via React Context + useReducer	ThemeContext , HistoryContext , SavedContext
Business	Domain logic, hooks, data transformation	useAudio.ts , useDebounce.ts , apiParser.ts
Data	HTTP communication, local persistence	api.ts , dictionaryService.ts , AsyncStorage
External	Third-party services	Dictionary API, Audio CDN

4. Navigation Flow

Shows every possible user navigation path through the application.

```
stateDiagram-v2
    [*] --> SplashScreen: App Launch

    SplashScreen --> Onboarding: First launch
    SplashScreen --> MainDrawer: Returning user

    state Onboarding {
        Step1: Intelligence
        Step2: Pronunciation
        Step3: Learn Anywhere
        Step4: Ready to Begin
        Step1 --> Step2: Next
        Step2 --> Step3: Next
        Step3 --> Step4: Next
        Step1 --> Step4: Skip
        Step2 --> Step4: Skip
        Step3 --> Step4: Skip
    }

    Onboarding --> MainDrawer: Get Started

    state MainDrawer {
        state DrawerMenu {
            Home
            SavedWords
            SearchHistory
            WordOfTheDay
            Settings
            About
            Feedback
        }

        Home --> WordDetails: Search / Tap trending
        SavedWords --> WordDetails: Tap saved word
        SearchHistory --> WordDetails: Tap history item
        DrawerMenu --> Home: Navigate
        DrawerMenu --> SavedWords: Navigate
        DrawerMenu --> SearchHistory: Navigate
        DrawerMenu --> Settings: Navigate
    }

    state WordDetails {
        WordHeader: Word + Phonetics
        Meanings: Definitions + Examples
        Synonyms: Synonyms & Antonyms
        AudioPlay: Pronunciation Audio

        WordHeader --> AudioPlay: Tap speaker icon
        WordHeader --> Meanings: Scroll down
        Meanings --> Synonyms: Scroll down
    }

    WordDetails --> MainDrawer: Back button

    state ErrorStates {
        NotFound: Word Not Found (404)
        Offline: Connection Lost
        Timeout: Server Timeout
    }

    WordDetails --> ErrorStates: API Error
    ErrorStates --> Home: Return to Explore
    ErrorStates --> WordDetails: Retry
    ErrorStates --> SavedWords: Access Saved Words
```

5. API Endpoints

Base URL

```
https://api.dictionaryapi.dev/api/v2/entries/en/
```

Endpoint Specification

```
sequenceDiagram
    participant U as 👤 User
    participant A as 📱 Verba App
    participant V as ✅ Validator
    participant X as 📡 Axios Client
    participant API as 🌐 Dictionary API
    participant CDN as 🎧 Audio CDN

    Note over U,CDN: Activity 1: Word Search & API Integration

    U->>A: Type word + tap Search
    A->>V: Validate input
    alt Empty input
        V-->>A: ❌ EMPTY_QUERY
        A-->>U: Alert "Please enter a word"
    else Valid input
        V-->>A: ✅ Formatted word
    end

    A->>X: lookupWord(word)
    X->>API: GET /api/v2/entries/en/{word}
    Note right of X: Headers: Accept: application/json<br/>Timeout: 8000ms

    alt 200 OK
        API-->>X: JSON Array [DictionaryEntry]
        X-->>A: Parsed DictionaryEntry
        Note over A: Activity 2: Display Details
        A->>U: Show word, phonetics,<br/>meanings, definitions, examples
        A->>A: Add to SearchHistory
    else 404 Not Found
        API-->>X: { title, message, resolution }
        X-->>A: throw WORD_NOT_FOUND
        Note over A: Activity 5: Error Handling
        A->>U: Show "Word Not Found" screen
    else Network Error
        X-->>A: throw NETWORK_TIMEOUT
        A->>U: Show "Connection Lost" screen
    end

    Note over U,CDN: Activity 3: Audio Pronunciation

    U->>A: Tap speaker icon 🎧
    A->>CDN: GET {phonetics[.audio]}
    Note right of A: URL example:<br/>https://api.dictionaryapi.dev/<br/>media/pronunciations/en/<br/>hello-us.mp3
    CDN-->>A: MP3 audio stream
    A-->>U: 🎧 Play pronunciation
```

API Request & Response Reference

Request

Field	Value
Method	GET
URL	https://api.dictionaryapi.dev/api/v2/entries/en/{word}
Headers	Accept: application/json , Content-Type: application/json
Timeout	8000ms
Library	Axios (as required by Integrated Situation)

Success Response (200 OK)

```
[
  {
    "word": "hello",
    "phonetic": "/hə'lou/",
    "phonetics": [
      {
        "text": "/hə'lou/",
        "audio": "https://api.dictionaryapi.dev/media/pronunciations/en/hello-us.mp3",
        "sourceUrl": "https://commons.wikimedia.org/..."
      }
    ],
    "meanings": [
      {
        "partOfSpeech": "noun",
        "definitions": [
          {
            "definition": "\"Hello!\" or an equivalent greeting.",
            "synonyms": ["greeting"],
            "antonyms": [],
            "example": "She began the conversation with a hello."
          }
        ],
        "synonyms": ["greeting"],
        "antonyms": []
      }
    ],
    "sourceUrls": ["https://en.wiktionary.org/wiki/hello"]
  }
]
```

Error Response (404 Not Found)

```
{
  "title": "No Definitions Found",
  "message": "Sorry pal, we couldn't find definitions for the word you were looking for.",
  "resolution": "You can try the search again at a later time or head to the web instead."
}
```

Error Code Mapping

HTTP Status	Axios Error	App Error Code	User-Facing Message
200	—	—	Display results
404	<code>error.response.status === 404</code>	<code>WORD_NOT_FOUND</code>	"Linguistic Discovery Pending"
5xx	<code>error.response</code> (non-404)	<code>SERVER_ERROR</code>	"Connection Lost"
No response	<code>error.request</code> (timeout)	<code>NETWORK_TIMEOUT</code>	"You're offline"
Parse fail	<code>catch</code>	<code>UNKNOWN_ERROR</code>	"Something went wrong"
Empty input	Validation	<code>EMPTY_QUERY</code>	Alert: "Please enter a word"

6. Required Pages / Screens

Screen Inventory

```
graph TB
  subgraph Entry["🔑 Entry Flow"]
    Splash["SplashScreen\n🌀 Loading check"]
    Onboard["OnboardingScreen\n📖 4-step carousel"]
  end

  subgraph Core["📁 Core Screens (Drawer)"]
    Home["DiscoverScreen\n🏠 Home / Search"]
    Details["WordDetailsScreen\n📖 Definitions"]
    History["HistoryScreen\n🕒 Search History"]
    Saved["SavedWordsScreen\n⭐ Bookmarked Words"]
    Settings["SettingsScreen\n⚙️ Preferences"]
  end

  subgraph States["⚠️ System States"]
    E404["ErrorView: 404\n🔍 Word Not Found"]
    EOffline["ErrorView: Offline\n📶 No Connection"]
    ETimeout["ErrorView: Timeout\n⌚ Server Error"]
    Empty["EmptyState\n📄 No Data"]
    Loading["LoadingSpinner\n⌛ Fetching"]
  end

  Splash --> Onboard
  Splash --> Home
  Onboard --> Home
  Home --> Details
  History --> Details
  Saved --> Details
  Details --> E404
  Details --> EOffline
  Details --> ETimeout
  Details --> Loading

  style Entry fill:#F5F3FF,stroke:#7C3AED,stroke-width:2px
  style Core fill:#EFF6FF,stroke:#2563EB,stroke-width:2px
  style States fill:#FEEF2F,stroke:#DC2626,stroke-width:2px
```


Screen Details Table

#	Screen	Route	Purpose	Activity	Key Components
1	SplashScreen	— (initial)	Brand loading + first-launch check via AsyncStorage	—	Logo, progress indicator
2	OnboardingScreen	Onboarding	4-step horizontal carousel introducing app features	—	Carousel, dot indicators, Skip/Next/Get Started buttons
3	DiscoverScreen	Discover	Home screen with search bar, WotD card, trending words, bento grid stats	Act 1, 4	SearchBar, WordCard, suggestions panel
4	WordDetailsScreen	WordDetails	Full word display: phonetics, meanings, definitions, examples, audio, synonyms/antonyms	Act 2, 3	MeaningCard, DefinitionItem, ExampleText, PronunciationButton
5	HistoryScreen	History	Chronological list of all previously searched words	Act 4	HistoryItem, search/filter, clear all
6	SavedWordsScreen	SavedWords	Bookmarked words organized by collection, quiz feature	Extended	WordCard (mini), collection tabs, quiz modal
7	SettingsScreen	Settings	Theme toggle, font size, notifications, autoplay, data management	Extended	Toggle switches, sliders, action buttons

Screen-to-Activity Mapping

Activity	Primary Screen	Supporting Screens
Activity 1: Word Search & API Integration	DiscoverScreen	WordDetailsScreen
Activity 2: Display Word Details	WordDetailsScreen	—
Activity 3: Audio Pronunciation	WordDetailsScreen	DiscoverScreen (WotD audio)
Activity 4: Drawer Navigation & History	DrawerContent, HistoryScreen	DiscoverScreen (recent chips)
Activity 5: Error Handling & Feedback	ErrorView component	WordDetailsScreen, DiscoverScreen

7. Component Hierarchy

```
graph TD
    App["App.tsx"]
    App --> NavContainer["NavigationContainer"]
    NavContainer --> ThemeProvider["ThemeProvider"]
    ThemeProvider --> HistoryProvider["HistoryProvider"]
    HistoryProvider --> SavedProvider["SavedProvider"]
    SavedProvider --> AppNavigator["AppNavigator"]

    AppNavigator --> Stack["Stack Navigator"]
    Stack --> Onboarding["OnboardingScreen"]
    Stack --> DrawerNav["Drawer Navigator"]
    Stack --> WordDetails["WordDetailsScreen"]

    DrawerNav --> DrawerContent["DrawerContent\n(custom sidebar)"]
    DrawerNav --> Discover["DiscoverScreen"]
    DrawerNav --> SavedWords["SavedWordsScreen"]
    DrawerNav --> History["HistoryScreen"]
    DrawerNav --> Settings["SettingsScreen"]

    Discover --> SearchBar["SearchBar"]
    Discover --> WordCard["WordCard"]
    Discover --> SuggestionsPanel["SuggestionsPanel\n(inline)"]

    WordDetails --> MeaningCard["MeaningCard"]
    WordDetails --> PronBtn["PronunciationButton"]
    WordDetails --> ErrorView["ErrorView"]
    WordDetails --> LoadingSpin["LoadingSpinner"]

    MeaningCard --> DefItem["DefinitionItem"]
    DefItem --> ExText["ExampleText"]

    SavedWords --> EmptyState["EmptyState"]
    SavedWords --> SearchBar2["SearchBar"]

    style App fill:#4F46E5,color:#fff
    style AppNavigator fill:#7C3AED,color:#fff
    style DrawerNav fill:#0891B2,color:#fff
    style Stack fill:#0891B2,color:#fff
```

8. Data Models (Entity Relationship)

```
erDiagram
    DictionaryEntry {
        string word PK
        string phonetic
        string origin
        string[] sourceUrls
    }

    Phonetic {
        string text
        string audio
        string sourceUrl
    }

    Meaning {
        string partOfSpeech
        string[] synonyms
        string[] antonyms
    }

    Definition {
        string definition
        string example
        string[] synonyms
        string[] antonyms
    }

    SearchHistoryItem {
        string id PK
        string word
        string partOfSpeech
        string definitionSummary
        number timestamp
    }

    SavedWordItem {
        string word PK
        string phonetic
        string partOfSpeech
        string definitionSummary
        number timestamp
        string collection
        number masteryLevel
        string learningNotes
    }

    DictionaryEntry ||--o{ Phonetic : "has phonetics"
    DictionaryEntry ||--o{ Meaning : "has meanings"
    Meaning ||--o{ Definition : "has definitions"
    DictionaryEntry ||--o| SearchHistoryItem : "logged as"
    DictionaryEntry ||--o| SavedWordItem : "bookmarked as"
```

Type Definitions (TypeScript)

```
// From src/models/DictionaryTypes.ts

interface Phonetic {
  text: string;
  audio: string;
  sourceUrl?: string;
}

interface Definition {
  definition: string;
  synonyms: string[];
  antonyms: string[];
  example?: string;
}

interface Meaning {
  partOfSpeech: string;
  definitions: Definition[];
  synonyms: string[];
  antonyms: string[];
}

interface DictionaryEntry {
  word: string;
  phonetic: string;
  phonetics: Phonetic[];
  meanings: Meaning[];
  origin?: string;
  sourceUrls: string[];
}

interface SearchHistoryItem {
  id: string;           // UUID
  word: string;
  partOfSpeech: string;
  definitionSummary: string;
  timestamp: number;    // Date.now()
}

interface SavedWordItem {
  word: string;
  phonetic: string;
  partOfSpeech: string;
  definitionSummary: string;
  timestamp: number;
  collection: 'Favorites' | 'Academic' | 'Travel';
  masteryLevel: 1 | 2 | 3;
  learningNotes?: string;
}
```

9. State Management Architecture

```
graph TB
    subgraph Global["🌐 Global State (React Context)"]
        direction TB
        Theme["ThemeContext"]
        ThemeState["• isDark: boolean\n• themeColors: ColorPalette\n• fontSizeMultiplier: number\n• toggleTheme()\n• setFontSize()"]
        Theme --> ThemeState

        HistCtx["HistoryContext"]
        HistState["• history: SearchHistoryItem[]\n• addHistoryWord(word, pos, def)\n• clearHistory()\n• removeHistoryItem(id)"]
        HistCtx --> HistState

        SavedCtx["SavedContext"]
        SavedState["• savedWords: SavedWordItem[]\n• streakCount: number\n• addSavedWord(...)\n• removeSavedWord(word)\n• updateMastery"]
        SavedCtx --> SavedState
    end

    subgraph Local["🔴 Local State (useState)"]
        direction TB
        SearchState["DiscoverScreen\n• searchQuery: string\n• suggestions: string[]\n• debouncedQuery: string"]
        DetailsState["WordDetailsScreen\n• wordData: DictionaryEntry\n• isLoading: boolean\n• error: string | null"]
        AudioState["useAudio Hook\n• isPlaying: boolean\n• isPaused: boolean\n• isLoading: boolean"]
    end

    subgraph Persist["💾 Persistence (AsyncStorage)"]
        direction TB
        Keys["Keys:\n• verba_first_launch_done\n• verba_settings\n• verba_history\n• verba_saved_words"]
    end

    Global --> Local
    Global --> Persist

    style Global fill:#F0FDF4,stroke:#16A34A,stroke-width:2px,color:#000
    style Local fill:#FFF6FF,stroke:#2563EB,stroke-width:2px,color:#000
    style Persist fill:#FFF7ED,stroke:#EA580C,stroke-width:2px,color:#000
```

10. Error Handling Flow

flowchart TD

Start([User submits search]) --> Validate{Input empty?}

Validate -->|Yes| AlertEmpty[Alert: 'Please enter a word'\n🚫 Stays on current screen]

Validate -->|No| ShowLoader[Show LoadingSpinner\n⌚ Disable search button]

ShowLoader --> APICall["Axios GET\n/api/v2/entries/en/{word}"]

APICall --> CheckResponse{Response?}

CheckResponse -->|"200 OK"| Parse["Parse JSON via apiParser\n✅ Extract DictionaryEntry"]

Parse --> AddHistory["Add to SearchHistory\n📄 Prevent duplicates"]

AddHistory --> Display["Render WordDetailsScreen\n🗨 Word + Meanings + Audio"]

CheckResponse -->|"404"| Error404["throw WORD_NOT_FOUND\n🔍"]

Error404 --> Show404["ErrorView type='404'\n'Linguistic Discovery Pending'\n+ Request Word button\n+ Return to Explore link"]

CheckResponse -->|"error.request\n(no response)"| ErrorNetwork["throw NETWORK_TIMEOUT\n🌐"]

ErrorNetwork --> ShowOffline["ErrorView type='offline'\n'You're offline'\n+ Try Again button\n+ Access Saved Words link"]

CheckResponse -->|"error.response\n(5xx)"| ErrorServer["throw SERVER_ERROR\n🔥"]

ErrorServer --> ShowTimeout["ErrorView type='timeout'\n'Connection Lost'\n+ Error Code: API_TIMEOUT_503\n+ Retry Connection button"]

CheckResponse -->|"catch other"| ErrorUnknown["throw UNKNOWN_ERROR"]

ErrorUnknown --> ShowGeneric["Generic error message\n+ Retry button"]

Show404 -->|"Retry"| Start

ShowOffline -->|"Retry"| Start

ShowTimeout -->|"Retry"| Start

ShowGeneric -->|"Retry"| Start

style Start fill:#4F46E5,color:#fff

style Display fill:#16A34A,color:#fff

style Show404 fill:#F59E0B,color:#000

style ShowOffline fill:#EF4444,color:#fff

style ShowTimeout fill:#EF4444,color:#fff

style ShowGeneric fill:#6B7280,color:#fff

Directory Structure

```
D:\Mobile\Verba-Mobile\
├─ App.tsx                # Root component with providers
├─ package.json           # Dependencies (expo, axios, react-navigation)
├─ babel.config.js        # Babel with reanimated plugin
├─ app.json               # Expo configuration
├─ Integrated Situation.md # Requirements document
├─
├─ src/
│   ├── assets/           # Static assets (fonts, images)
│   ├── components/       # Reusable UI components
│   │   ├── SearchBar.tsx # Text input with clear/submit
│   │   ├── WordCard.tsx  # WotD hero card (word, phonetic, audio, definition)
│   │   ├── MeaningCard.tsx # Part-of-speech card with definitions
│   │   ├── DefinitionItem.tsx # Single definition with left-bar indicator
│   │   ├── ExampleText.tsx # Italic example sentence
│   │   ├── PronunciationButton.tsx # Audio play/pause button (expo-av)
│   │   ├── ErrorView.tsx  # Error states (404, offline, timeout)
│   │   ├── EmptyState.tsx # Empty collection / no results
│   │   ├── LoadingSpinner.tsx # Animated loading indicator
│   │   └── HistoryItem.tsx # History list row (swipeable)
│   ├── screens/          # Full-page screen components
│   │   ├── SplashScreen.tsx # Brand intro + first-launch check
│   │   ├── OnboardingScreen.tsx # 4-step feature carousel
│   │   ├── DiscoverScreen.tsx # Home: search, WotD, trending, stats
│   │   ├── WordDetailsScreen.tsx # Full word detail view
│   │   ├── HistoryScreen.tsx # Search history list
│   │   ├── SavedWordsScreen.tsx # Bookmarked words + quiz
│   │   └── SettingsScreen.tsx # App preferences
│   ├── navigation/       # Navigation configuration
│   │   ├── AppNavigator.tsx # Stack + Drawer navigator setup
│   │   └── DrawerContent.tsx # Custom drawer sidebar
│   ├── context/          # React Context providers
│   │   ├── ThemeContext.tsx # Dark/light mode + font scaling
│   │   ├── HistoryContext.tsx # Search history state
│   │   └── SavedContext.tsx # Saved words + streak state
│   ├── hooks/            # Custom React hooks
│   │   ├── useAudio.ts    # Audio playback via expo-av
│   │   └── useDebounce.ts # Input debouncing (200ms)
│   ├── services/         # Data access layer
│   │   ├── api.ts         # Axios instance (baseUrl, timeout, headers)
│   │   └── dictionaryService.ts # lookupWord() with error classification
│   ├── models/           # TypeScript type definitions
│   │   └── DictionaryTypes.ts # All interfaces
│   ├── utils/            # Utility functions
│   │   └── apiParser.ts   # Raw API JSON → DictionaryEntry
│   ├── styles/           # Design system
│   │   └── theme.ts       # Colors, spacing, typography tokens
```

Rendering These Diagrams

All diagrams above are written in **Mermaid** syntax. To render them:

Option 1: Mermaid Live Editor (Recommended)

1. Visit mermaid.live
2. Paste any code block between ````mermaid` and `````
3. The diagram renders instantly in the browser
4. Export as **SVG** or **PNG**

Option 2: VS Code Extension

1. Install “**Markdown Preview Mermaid Support**” extension
2. Open this `.md` file in VS Code
3. Press `Ctrl+Shift+V` to preview

Option 3: GitHub / GitLab

- Push this file to any GitHub repository — GitHub natively renders Mermaid in `.md` files

Option 4: Command Line (CLI)

```
npm install -g @mermaid-js/mermaid-cli
mmdc -i design_documentation.md -o output.pdf
```

Document Version: 1.0 **Author:** LexiTech Solutions Ltd — Mobile Development Team **Date:** June 8, 2026 **Referenced:** Integrated [Situation.md](#) (Assessment Brief)